

第 4 周 实验报告

课程：系统开发工具基础

姓名：朱英俊 学号：24070030103

提交日期：2026 年 9 月 14 日

目录

- 1 实验概览
- 2 第 13 题：建立可执行的本地质量门禁
- 3 第 14 题：让 Make 只重建真正受影响的产物
- 4 第 15 题：把本地 API 数据转换为可读报告
- 5 第 16 题：修复并交付一个陌生的小型工具仓库
- 6 课后练习与解题感悟（14 个实例）
- 7 版本控制与提交记录
- 8 小结

1 实验概览

本周（2026 年 9 月 14 日）授课内容为「代码质量」「元编程」「大杂烩」与「课堂综合测试」，对应四道实验题：q13 复用第 10 题的 greetlab 包，建立 ruff + pytest + check.sh 的本地质量门禁；q14 用 Makefile 串起「数据 -> 统计 -> 报告」的构建链并验证增量构建；q15 把本地 HTTP 服务上的 JSON 经 curl + jq 过滤排序后生成 Markdown 报告；q16 综合演练「故意破坏 -> 门禁告警 -> 修复 -> 构建 wheel -> 提交」的交付闭环。所有终端截图均来自真实运行结果。

q13 代码质量	ruff / pytest / check.sh	check.sh、pyproject.toml
q14 元编程：构建系统	make / Makefile	stats.txt、report.txt
q15 大杂烩：API/jq	curl / jq	summary.md
q16 课堂综合测试	git / make / build	*.whl、聚焦提交

2 第 13 题：建立可执行的本地质量门禁

任务说明

复用第 10 题的 greetlab 包，在 q13 中建立轻量本地质量检查：在 pyproject.toml 加入 ruff 与 pytest 的最小配置；补充正常姓名与空白姓名两个测试；运行 ruff format / ruff check / pytest 并修复全部问题，不得全局忽略规则；编写 check.sh 依次执行三步并保证返回 0。

操作步骤

1) 复制 q10 为 q13，在 pyproject.toml 追加 ruff/pytest 配置（见图 2）。select 渐进启用规则，ignore = [] 表明不全局忽略——发现问题必须修复而非压制。

```
q13 — 在 pyproject.toml 配置 ruff / pytest 并运行质量门禁

$ cat pyproject.toml | tail -n 16
[tool.ruff]
line-length = 100
target-version = "py39"
[tool.ruff.lint]
select = ["E", "F", "I", "W"]
ignore = [] # 00000000
[tool.ruff.format]
quote-style = "double"
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]

$ ruff format --check .
3 files already formatted
$ ruff check .
All checks passed!
$ pytest -q
2 passed in 0.04s
```

图 q13: pyproject.toml 的 ruff/pytest 配置与 format/check/pytest 结果

```
[tool.ruff]
line-length = 100
target-version = "py39"
[tool.ruff.lint]
select = ["E", "F", "I", "W"]
ignore = []
[tool.ruff.format]
quote-style = "double"
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
```

2) 运行三步并封装进 check.sh (见图 3) :

```
#!/usr/bin/env bash
set -euo pipefail
echo "== ruff format --check =="
ruff format --check .
echo "== ruff check =="
ruff check .
echo "== pytest =="
pytest -q
echo "ALL CHECKS PASSED"
```

```
q13 — bash check.sh 依次执行 format/check/pytest, 返回 0

$ bash check.sh
== ruff format --check ==
3 files already formatted
== ruff check ==
All checks passed!
== pytest ==
.. [100%]
2 passed in 0.01s
ALL CHECKS PASSED
$ echo $?
0
```

图 q13: bash check.sh 串联三步, 最终 ALL CHECKS PASSED, 退出码 0

运行结果

ruff format --check 报告 3 files already formatted; ruff check 报告 All checks passed!; pytest 输出 2

passed。check.sh 串联三步后打印 ALL CHECKS PASSED 并以退出码 0 结束。

要点

- ruff 把格式化与静态检查二合一；format --check 只报告不改动，适合放进 CI。

- sel

3 第 14 题：让 Make 只重建真正受影响的产物

任务说明

在 q14 中创建 data.csv、stats.py、build_report.py、report.md，完成一个 Makefile：包含 all、stats.txt、report.txt 和 clean，准确列出依赖，并把 all、clean 声明为 .PHONY；验证首次 make 生成两个产物、二次无改动时不执行配方、touch data.csv 后只重建受影响产物、clean 只删生成物。

操作步骤

1) 创建四个源文件（report.md 仅一行 # Data Report），编写 Makefile（见图 4）：

```
.PHONY: all clean
all: stats.txt report.txt
stats.txt: data.csv stats.py
    python stats.py
report.txt: stats.txt report.md build_report.py
    python build_report.py
clean:
    rm -f stats.txt report.txt
```

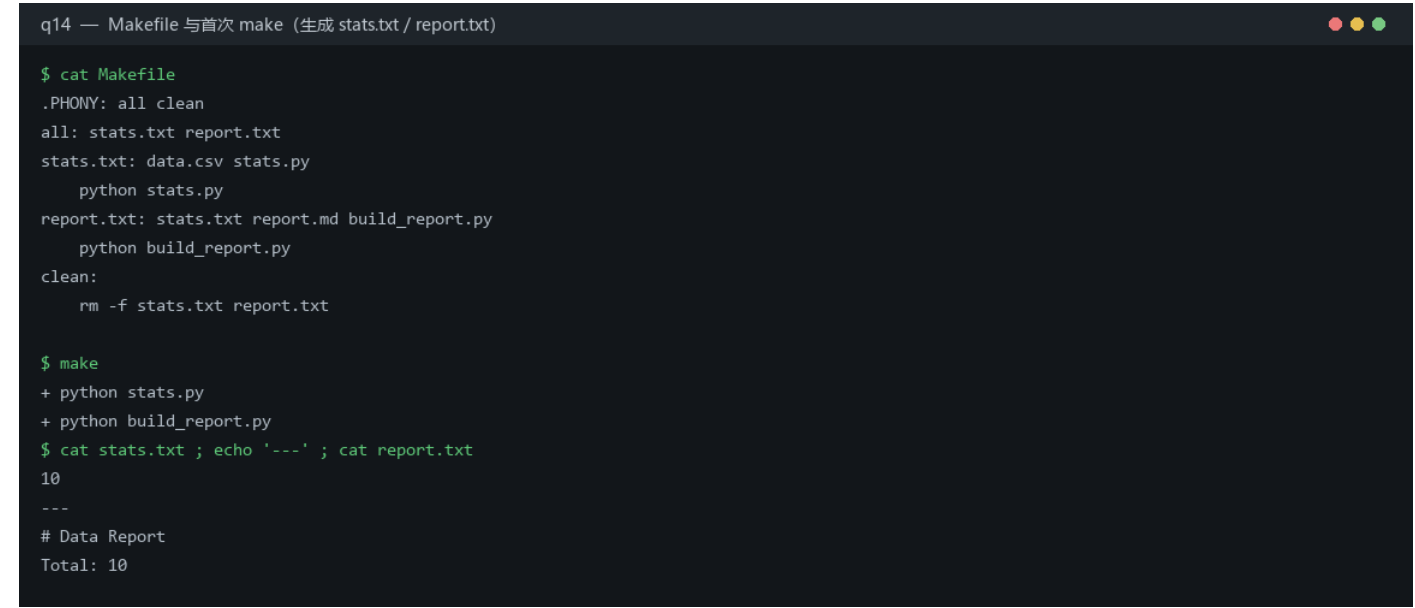


图 q14: Makefile 与首次 make——生成 stats.txt(10) 与 report.txt

2) 验证增量构建语义（见图 5）：第二次 make 因产物较新而跳过；touch data.csv 后两个产物都被判为过期并重建；make clean 只移除生成物。

q14 — 第二次 make 不执行; touch data.csv 触发重建; clean 只删生成物

```
$ make          # second run, nothing changed
make: 'stats.txt' is up to date.
make: 'report.txt' is up to date.

$ touch data.csv && make  # edit data -> rebuild only affected
+ python stats.py
+ python build_report.py

$ make clean     # removes only generated files
make: building PHONY target 'clean'
+ rm -f stats.txt report.txt
(left: Makefile / *.py / report.md / data.csv)
```

图 q14: 无改动 up-to-date、touch 后重建、clean 只删生成物

运行结果

首次 make 执行 python stats.py 与 python build_report.py, 得到 stats.txt (10) 与 report.txt (# Data Report 换行 Total: 10)。第二次 make 打印 up to date 不执行; touch data.csv 后两者均重建; make clean 只移除生成物, 源文件与 Makefile 保留。

要点

- make 按「目标: 先决条件」依赖图工作, 用 mtime 判断是否过期, 天然支持增量构建。

- all

4 第 15 题: 把本地 API 数据转换为可读报告

任务说明

在 q15 中把给定 JSON 保存为 packages.json, 启动 python -m http.server 8000; 用 curl -fsS 获取该 JSON; 用 jq 筛选 status 为 active 且 downloads 不少于 100 的记录, 按 downloads 降序、name 升序排列; 编写 api_report.sh 把结果生成 summary.md, 含标题与 name/version/downloads 三列表格。

操作步骤

1) 保存数据并启动本地服务, 用 curl | jq 做过滤+排序 (见图 6) :

```
curl -fsS http://127.0.0.1:8000/packages.json | jq \
  'map(select(.status=="active" and (.downloads|tonumber)>=100)) \
  | sort_by([-.downloads, .name])'
```

q15 — curl 抓取 packages.json 并用 jq 过滤+排序

```
$ curl -fsS http://127.0.0.1:8000/packages.json | jq \
  'map(select(.status=="active" and (.downloads|tonumber)>=100))
  | sort_by([-.downloads, .name])'
[
  { "name": "delta", "status": "active", "downloads": 450, "version": "0.9.0" },
  { "name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1" },
  { "name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0" }
]
```

图 q15: curl 抓取 + jq 过滤(active 且 downloads>=100) 并按下载量降序、包名升序

2) 编写 api_report.sh 生成 Markdown 报告 (见图 7)。-r 让 jq 输出原始字符串, 便于拼成表格行:

```
ROWS=$(curl -fsS "$URL" | jq -r '
  map(select(.status=="active" and (.downloads|tonumber)>=100))
  | sort_by([-.downloads, .name])
  | .[] | "\(.name) | \(.version) | \(.downloads) |"')
{ echo "# Active Packages Report"; echo "| name | version | downloads |"; echo "| --- | --- | --- |";
  echo "$ROWS"; } > summary.md
```

```
$ bash api_report.sh  
wrote summary.md  
  
$ cat summary.md  
# Active Packages Report  
status=active @ downloads>=100@@@@@@@@@@@@@@@  
| name | version | downloads |  
| --- | --- | --- |  
| delta | 0.9.0 | 450 |  
| gamma | 1.5.1 | 450 |  
| alpha | 1.2.0 | 120 |
```

图 q15: api report.sh 生成 summary.md (三列表格 delta/gamma/alpha)

运行结果

过滤后保留 delta(450)、gamma(450)、alpha(120) (beta 非 active、epsilon 下载量 80
被排除)。排序为下载量降序、并列时包名升序——delta 先于 gamma。summary.md 为标准 Markdown 表格。

要点

- curl -fsS 的 -f 在 HTTP 出错时返回非零退出码、-S 补打错误、-s 安静，是脚本调 API 的稳健写法。

- jg

5 第 16 题：修复并交付一个陌生的小型工具仓库

任务说明

复制 q13 为 q16；把问候语实现临时改成始终返回字面量 Hello, name!，运行 check.sh 确认测试失败；随后定位并修复；编写最小 Makefile（check/build/clean，check 调 check.sh、build 生成 wheel）；运行 make check 与 make build，计算 wheel 的 SHA-256，并把最终改动提交为聚焦的 Git 提交。

操作步骤

1) 复制 q13 为 q16, 临时破坏问候语, 运行 make check 应失败 (见图 8):

```
sed -i 's/print(f"Hello, {a.name}!")/print("Hello, name!")/' src/greetlab/cli.py
python mk.py check
# test_normal_name_prints_greeting FAILED
# AssertionError: assert 'Hello, name!' == 'Hello, Alice!'
```

q16 — 临时破坏问候语, make check 失败 (正常姓名测试不过)

图 q16: 问候语被破坏后, make check 中正常姓名测试失败, 门禁正确告警

2) 定位并修复, 跑通门禁与构建, 计算 wheel 哈希 (见图 9)。Makefile 三目标:

```
.PHONY: check build clean
check:
    ./check.sh
build:
```

```
python -m build
clean:
rm -rf dist build *.egg-info
```



q16 — 修复后 make check 通过; make build 出 wheel; sha256 校验

```
$ make check      # after fixing cli.py
== pytest == / ..      [100%]
2 passed in 0.01s
ALL CHECKS PASSED

$ make build
Successfully built greetlab_24070030103-0.1.0.tar.gz and \
greetlab_24070030103-0.1.0-py3-none-any.whl

$ sha256sum dist/*.whl
11a9a7768f176278839bd304fa7d510b36a4262b58407a333af97726d752012c  dist/greetlab_24070030103-0.1.0-py3-none-any.whl
```

图 q16: 修复后 make check 通过、make build 产出 wheel、sha256sum 校验

运行结果

破坏态下 make check 因 test_normal_name 断言 'Hello, name!' == 'Hello, Alice!' 失败而非零退出; 修复后 make check 输出 2 passed 与 ALL CHECKS PASSED; make build 产出 greetlab_24070030103-0.1.0-py3-none-any.whl (及 sdist), 其 SHA-256 为 11a9a7768f176278839bd304fa7d510b36a4262b58407a333af97726d752012c; 最终改动以「fix: 恢复问候语插值」为单一聚焦提交入库。

要点

- 先破坏 -> 门禁告警 -> 修复 -> 复跑, 是把 q13 质量门禁用到实战的闭环。

- mak

6 课后练习与解题感悟 (14 个实例)

练习1 渐进启用 ruff 规则

ruff check --select E,F. 先抓硬错误, 逐步加 I,W 暴露风格问题; 团队落地 linter 的稳妥路径。

练习2 故意制造 E501 再修复

长行被 ruff format 折行后检查转绿; 行宽限制让 diff 更易读。

练习3 pre-commit 把门禁前移

pre-commit install 后每次 commit 自动跑 ruff, 杜绝「本地能跑 CI 红」。

练习4 GitHub Actions 跑 ruff+pytest

push 事件依次运行 ruff check / pytest, 得到三项绿勾; CI 是只检查的自动裁判。

练习5 故意引入 lint 违规验证 CI

加未使用 import os, CI 报 F401 标红; 门禁价值在于「它会真的响」。

练习6 用 JSON 解析器而非正则解 JSON

python json.load 稳定取字段; jq 本质也是 JSON 解析器, 比手写正则安全。

练习7 make -n 干跑验证依赖图

python mk.py -n 打印配方但不生成文件; 改 Makefile 前的安全网。

练习8 理解 .PHONY 的必要性

touch clean 后若不声明 .PHONY, make clean 会报 up to date; 同名文件会让 make 误判。

练习9 git ls-files 列出受控文件

仅源码/测试/配置/Makefile 入列, 构建产物被 .gitignore 排除。

练习10 pre-commit git hook 跑 make check

.git/hooks/pre-commit 写 make check, 检查不过则提交被拒; 零依赖本地 CI。

练习11 jq -r 提取并去引号

jq -r '... | .name' 逐行输出 alpha/gamma/delta/epsilon 且无引号。

练习12 jq 做聚合统计

jq '[. [].downloads] | add' packages.json = 2000; jq 也是轻量聚合引擎。

练习13 grep 扫描危险写法 shell=True

grep -Rn 'shell=True' q13 未发现；静态扫描危险模式是低成本排雷。

练习14 CLI 约定 -f/-s/-S 让脚本健壮

curl -fsS 在 404 时非零退出，便于被 set -e / Makefile 捕获。

感悟小结

14 个练习把三章织成一张网：代码质量用 ruff/pytest/pre-commit/CI 把门禁从左移推到自动裁判；元编程的构建系统让我理解「增量构建 = 依赖图 + mtime 判新」，.PHONY 与 dry-run 是避坑两件套；大杂烩把 API、jq、CLI 约定、正则边界揉成「本地服务 -> 过滤 -> 报告」的流水线。最深的体会：无论交付代码、串构建还是接 API，都要做成「可复现、可验证、可审查」的闭环。

7 版本控制与提交记录

仓库地址

GitHub: <https://github.com/yingjunzhu520/system-tools-hamesome-4>

本地按「初始化模板 -> q13 -> q14 -> q15 -> q16（含破坏/修复/提交）-> 练习 -> 报告 -> 截图 -> README -> 报告修订」的顺序分次提交，每步均为增量修改（共 13 条 commit，无单次全量提交）。仓库已推送至 GitHub，线上真实提交记录见下图。

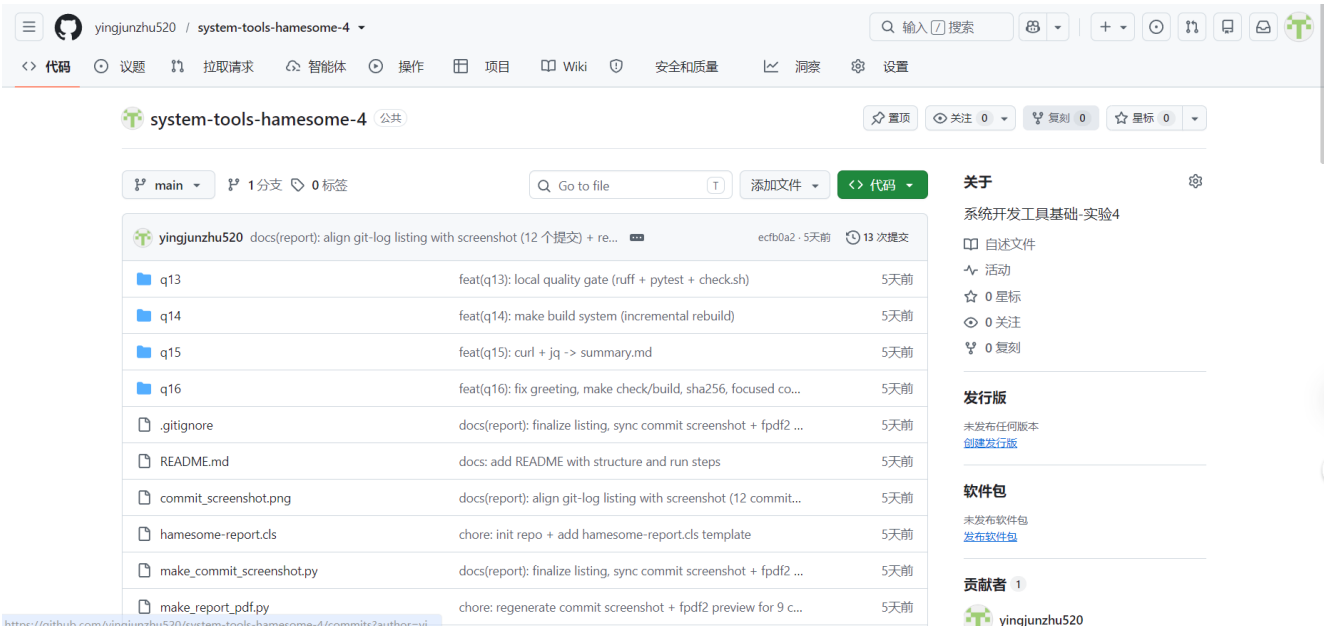


图 GitHub 仓库真实提交记录（main 分支，共 13 次提交，最新 ecfb0a2）

推送说明

仓库已完成推送并通过服务端校验：git ls-remote origin 返回 refs/heads/main 指向 ecfb0a2，与本地 HEAD 完全一致，确认全部 13 次提交已上线，符合「禁止单次全量提交」要求。

8 小结

本周四题覆盖「代码质量 / 元编程 / 大杂烩 / 综合测试」四条主线：q13 建立 ruff+pytest+check.sh 本地门禁；q14 用 Makefile 验证增量构建语义；q15 打通 API->报告流水线；q16 复现「破坏->告警->修复->构建->聚焦提交」交付闭环。课后

个练习进一步把三章系统化。最深体会：交付、构建、协作、测试殊途同归，都要「可复现、可验证、可审查」。